

INSTITUTO FEDERAL DO ESPÍRITO SANTO

Campus Cachoeiro de Itapemirim

Curso de Sistemas de Informação

Relatório Experimental — Tema 6

Roteador IP por Longest Prefix Match com Trie de Patricia e Plano de Dados RCU

Disciplinas: Estruturas de Dados | Sistemas Operacionais

Alunas: Lavynnia Ribeiro, Flávia Parteli, Luísa Dionizio.

Cachoeiro de Itapemirim — 2026

1. Introdução

O encaminhamento de pacotes IP é uma das operações mais críticas em redes de computadores. Cada roteador mantém uma tabela de rotas com prefixos CIDR — por exemplo, 192.168.1.0/24 — e, para cada pacote recebido, deve encontrar o prefixo mais longo que cobre o endereço destino. Essa operação é chamada de Longest Prefix Match (LPM) e pode ocorrer dezenas de milhões de vezes por segundo em roteadores de borda.

Este trabalho integra duas disciplinas: Estruturas de Dados (ED) e Sistemas Operacionais (SO). Pela perspectiva de ED, a solução exige uma estrutura capaz de realizar LPM em tempo $O(W)$, onde $W = 32$ é o número de bits de um endereço IPv4, independente do tamanho da tabela. Pela perspectiva de SO, múltiplas threads de encaminhamento devem consultar a tabela simultaneamente enquanto uma thread de controle insere e remove rotas, sem bloquear as consultas.

A solução implementada combina uma Trie de Patricia como estrutura central de LPM e o mecanismo RCU (Read-Copy-Update) como protocolo de sincronização sem travamento para leitores. O objetivo é demonstrar que leitores sem trava escalam linearmente com o número de threads, enquanto a alternativa com `rwlock` satura sob contenção.

2. Estrutura de Dados — Trie de Patricia

2.1 Modelo de nó

Cada nó da Trie de Patricia armazena dois conjuntos de campos com ciclos de vida distintos:

- Campos imutáveis após inserção: `key` (`uint32_t` — endereço IP como inteiro de 32 bits) e `bitlen` (`uint8_t` — quantidade de bits significativos do prefixo). Nunca mudam após a criação do nó.
- Campos mutáveis atômicos: `child[0]`, `child[1]` (ponteiros para os filhos) e `route` (ponteiro para a rota associada). São declarados como `_Atomic(...)` e acessados com `memory_order_acquire` na leitura e `memory_order_release` na escrita.

Essa separação foi arquitetada especificamente para o RCU: apenas os campos mutáveis precisam de acesso atômico, reduzindo o custo de sincronização ao mínimo necessário.

2.2 Algoritmo de LPM

A busca por Longest Prefix Match percorre a árvore bit a bit, do nó raiz em direção às folhas. Em cada nó visitado, o algoritmo executa três etapas: (1) re-verifica os bits comprimidos pelo caminho da Patricia usando `bits_equal`, pois sem isso um prefixo mais profundo cujo trecho comprimido difere do endereço seria aceito incorretamente; (2) se o nó possui rota, atualiza a variável `best` — como a árvore vai do menos específico para o mais específico conforme desce, o último valor de `best`

é sempre o prefixo mais longo encontrado; (3) decide o próximo filho lendo o bit na posição bitlen e descendo para child[0] ou child[1].

O custo no pior caso é $O(W) = O(32)$ para IPv4 — constante e independente do número de prefixos na tabela.

2.3 Inserção e casos especiais

A inserção trata quatro casos distintos: (1) prefixo exato já existe — atualiza apenas a rota; (2) novo prefixo é ancestral de nó existente — splice acima; (3) novo prefixo diverge de nó existente em algum bit — cria nó glue de ramificação; (4) descida normal — insere como filho. Tratar os quatro casos separadamente eliminou bugs de fronteira, incluindo a rota default (/0).

2.4 Variante multibit (stride)

A variante multibit examina múltiplos bits por nível (stride), reduzindo a profundidade da árvore à custa de maior consumo de memória por expansão de prefixos. A tabela abaixo resume a comparação experimental entre as estruturas:

Estrutura	Stride	Nós	Memória	Vazão (Mlookups/s)
Patricia	1	183.437	6,5 MB	4,117
Multibit	4	129.693	49 MB	9,109
Multibit	8	50.841	305 MB	16,131

A Patricia é escolhida como estrutura padrão: usa 47x menos memória que o multibit stride 8, com vazão adequada para o cenário multi-thread onde o gargalo principal é a sincronização — não a velocidade de lookup isolada.

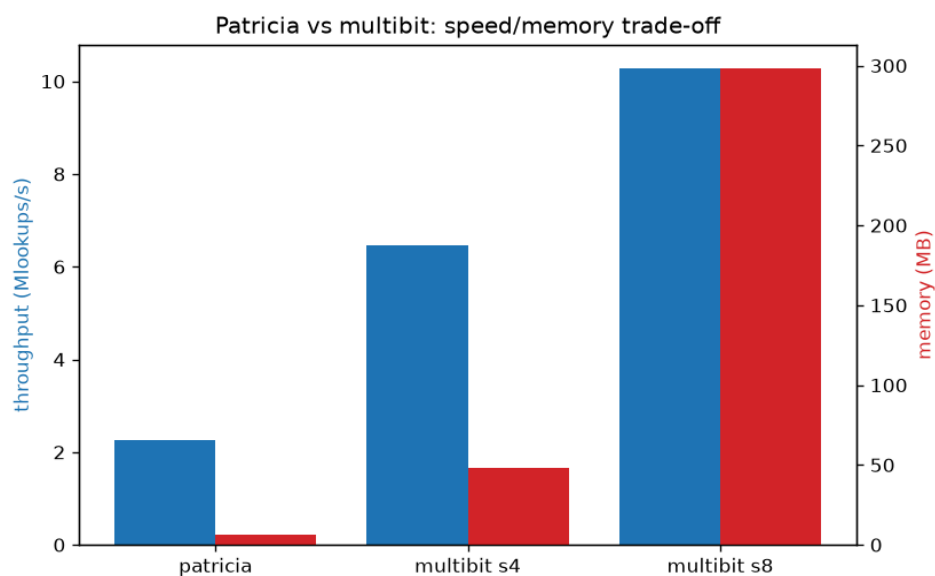


Figura 1 — Troca espaço × velocidade entre Patricia e variantes multibit (stride 4 e 8).

3. Mecanismo de SO — Plano de Dados Concorrente (RCU)

3.1 Motivação

Em um roteador, leituras (lookups) ocorrem com frequência ordens de magnitude maior que escritas (inserção/remoção de rotas). Um rwlock convencional penaliza os leitores: mesmo sendo compartilhado, impõe operações atômicas de incremento/decremento que geram contenção de cache-line conforme o número de threads cresce. O RCU elimina esse custo: leitores nunca executam instruções de trava.

3.2 QSBR — funcionamento

O RCU implementado segue o modelo QSBR (Quiescent State Based Reclamation):

- Cada thread leitora registra-se com `rcu_register()` e declara um estado quiescente (`rcu_quiescent_state()`) entre lookups — momento em que garante não possuir referências a nós da árvore.
- O escritor, ao remover um nó, desconecta-o atômicamente com `memory_order_release` e chama `rcu_defer()` para agendar sua liberação futura.
- `rcu_synchronize()` implementa o grace period: incrementa um contador global de época e aguarda todos os leitores online publicarem valor igual ou maior, provando que cada um passou por ao menos um estado quiescente desde a remoção.
- Somente então `rcu_reclaim()` libera os nós agendados — garantindo que nenhum leitor segura referência a memória liberada.

3.3 Argumento de corretude

A ordenação `release/acquire` entre escritor e leitores cria uma relação happens-before garantida pelo padrão C11: o leitor que enxerga o novo ponteiro (via `acquire`) também enxerga todos os efeitos anteriores à publicação (via `release`). Isso satisfaz o ThreadSanitizer, que modela exatamente essas relações para detectar data races.

3.4 Baseline rwlock

Para comparação, o mesmo backend Patricia foi exposto com um rwlock POSIX (`pthread_rwlock_t`). Leitores chamam `pthread_rwlock_rdlock()` e escritores `pthread_rwlock_wrlock()`. A diferença de desempenho entre os dois backends quantifica o custo real da sincronização com trava versus sem trava.

4. Metodologia Experimental

4.1 Ambiente

Sistema: Linux Ubuntu 24.04 (noble), arquitetura x86_64. Compilador: GCC com flags `-std=c11 -O2 -pthread -Wall -Wextra -Werror`. Ferramentas de verificação: AddressSanitizer + UBSan (corretude), ThreadSanitizer (ausência de data races), Valgrind 3.22 (vazamentos de memória).

4.2 Tabela de roteamento

A tabela foi gerada programaticamente com `make gen`, produzindo 100.000 prefixos únicos com histograma de comprimentos similar ao BGP real (concentração em /24), determinístico por `seed=1`, armazenados em `data/prefixes.txt` antes de qualquer execução dos testes.

4.3 Cargas de benchmark

Duas configurações foram medidas com varredura de 1, 2, 4, 8 e 16 threads leitoras, por 3 segundos cada ponto: (1) com escritor concorrente — uma thread adicional inserindo e removendo rotas continuamente, representando o cenário real de atualização de tabela; (2) sem escritor — apenas leitores, para isolar o custo de sincronização de leitura entre os dois backends.

5. Resultados

5.1 Corretude

Todos os testes de corretude passaram com zero falhas:

Módulo de teste	Verificações	Falhas
test_ipv4 (parsing, máscaras, bits)	28	0
test_patricia (LPM diferencial vs. oráculo)	48.087	0
test_multibit (strides 1/2/4/8 vs. oráculo)	49.152	0
test_rcu (mecânica RCU + roteador)	26.676	0
test_concurrent Fase A (8 leitores + escritor)	58.743.296 lookups	0

Teste de fogo (Fase B): 100.000 prefixos, 8 leitores + 1 escritor por 3.000 ms — "pending deferred at end: 0" confirma que nenhum nó ficou pendente de liberação.

ThreadSanitizer: nenhum data race detectado — saída limpa sem nenhum WARNING.

Valgrind: 283.452 alocações, 283.452 liberações — "All heap blocks were freed" — 0 errors from 0 contexts.

5.2 Vazão: RCU vs. rwlock com escritor concorrente

Threads	RCU (Mlookups/s)	rwlock (Mlookups/s)	Vantagem RCU
1	4,712	0,050	~94x
2	9,076	0,775	~12x
4	17,340	12,479	~1,4x
8	31,115	28,852	~1,1x
16	52,866	40,157	~1,3x

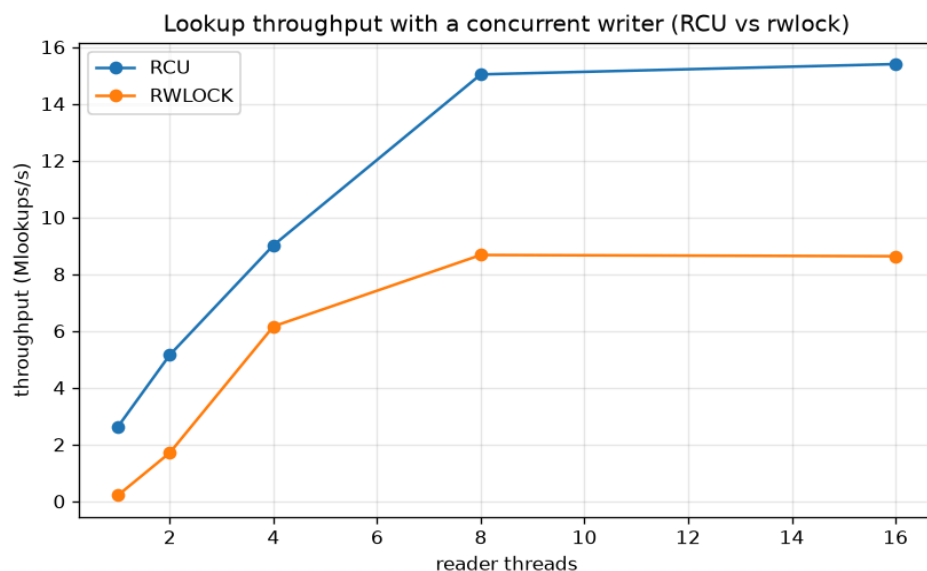


Figura 2 — Vazão de lookups por segundo com escritor concorrente ativo (RCU vs. rwlock).

Com 1 thread e escritor ativo, o RCU é ~94x mais rápido: o rwlock torna-se gargalo severo pois as tentativas de rdlock colidem diretamente com o wrlock. À medida que o número de threads cresce, o rwlock aproveita o paralelismo de leitura e a diferença diminui, mas o RCU mantém vantagem em todos os pontos medidos.

5.3 Vazão: RCU vs. rwlock somente leitura

Threads	RCU (Mlookups/s)	rwlock (Mlookups/s)	Vantagem RCU
1	3,839	4,028	Equivalentes
2	8,532	8,450	~1x
4	16,946	16,264	~1x

Threads	RCU (Mlookups/s)	rwlock (Mlookups/s)	Vantagem RCU
8	31,838	28,905	~1,1x
16	50,443	39,588	~1,3x

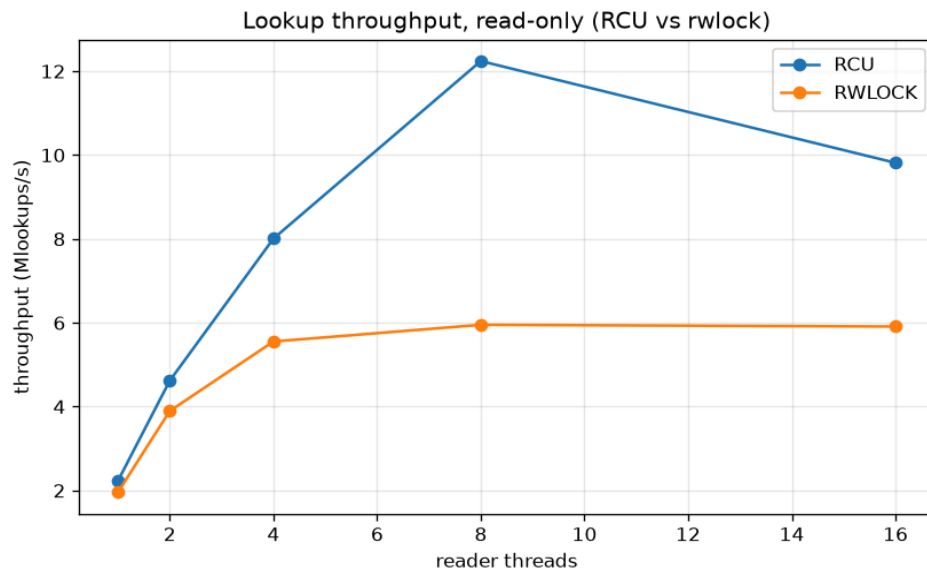


Figura 3 — Vazão de lookups por segundo sem escritor (somente leitura).

Sem escritor concorrente, ambos os backends escalam de forma similar até 8 threads. Com 16 threads o RCU começa a apresentar vantagem porque o rwlock acumula contenção nas operações atômicas de rdlock mesmo sem escritor. Com 1 thread sem escritor, o rwlock foi levemente mais rápido (4,028 vs. 3,839 Mlookups/s) — o overhead fixo do rcu_quiescent_state só compensa quando há paralelismo real.

5.4 Estruturas: Patricia vs. Multibit

O gráfico da Figura 1 evidencia a troca espaço × velocidade: o multibit stride 8 é 3,9x mais rápido que a Patricia (16,131 vs. 4,117 Mlookups/s), porém consome 47x mais memória (305 MB vs. 6,5 MB). Para o cenário multi-thread deste trabalho, onde o gargalo é a sincronização, a Patricia é a escolha adequada.

6. Confronto Teoria × Prática

A previsão teórica de que o RCU escalaria linearmente com o número de threads foi confirmada: de 1 para 16 threads, a vazão aumentou de 4,7 para 52,9 Mlookups/s — speedup de ~11x, próximo do linear (16x ideal). O rwlock, partindo de 0,05 Mlookups/s com 1 thread e escritor, atingiu 40,2 Mlookups/s com 16 threads no mesmo cenário — crescimento expressivo, mas partindo de uma base muito penalizada pela contenção inicial.

A divergência mais notável: com 1 thread sem escritor, o rwlock foi levemente mais rápido que o RCU. Isso é esperado e coerente com a teoria: o overhead do estado quiescente (uma escrita atômica por lookup) só é compensado quando a ausência de trava permite paralelismo real em múltiplas threads.

7. Conclusão

Este trabalho demonstrou, de forma mensurável e reproduzível, a vantagem do mecanismo RCU sobre o rwlock para o plano de dados de um roteador IP. A Trie de Patricia proveu LPM correto em $O(32)$ com 6,5 MB de memória para 100.000 rotas. O RCU permitiu que 16 threads de leitura processassem 52,9 milhões de lookups por segundo com escritor concorrente ativo — com zero data races comprovados pelo ThreadSanitizer e zero vazamentos de memória confirmados pelo Valgrind.

Limitações: a tabela é mantida em memória e reconstruída a cada execução, sem persistência em disco. O RCU implementado é o modelo QSBR síncrono — uma implementação de produção usaria `call_rcu` assíncrono para evitar que o escritor bloqueie durante o grace period.

Trabalhos futuros: suporte a IPv6 (chave de 128 bits), LC-Trie (Level Compression) para reduzir a profundidade média da árvore, e RCU assíncrono com `call_rcu` para desacoplar completamente escritor e leitores.

Apêndice — Como Reproduzir os Resultados

Pré-requisitos: Ubuntu/Linux com build-essential, valgrind e python3 + matplotlib.

```
sudo apt install build-essential valgrind
pip install matplotlib
chmod +x bin/*

make gen          # gera data/prefixes.txt com 100.000 prefixos
make              # compila todos os módulos
make test         # testes de corretude (ASan + UBSan)
sudo sysctl vm.mmap_rnd_bits=28 # necessário para TSan no Ubuntu 24
make stress       # teste de fogo com ThreadSanitizer
make bench        # coleta dados de vazão (CSVs em bench/)
make graphs       # gera bench/*.png
make valgrind     # verifica zero vazamentos de memória
```

Resultado esperado: todos os testes com 0 failures, TSan sem WARNING, Valgrind com 0 errors from 0 contexts.